

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: February 19, 2026

Lab Task # 02

S-DES Algorithm Implementation

1. Code

```
#include <iostream>
#include <string>
#include <bitset>
#include <cstdint>

using namespace std;

// Permutation Tables
// All indices are 1-based (as in the SDES specification)
const int P10[10] = {3, 5, 2, 7, 4, 10, 1, 9, 8, 6};
const int P8[8] = {6, 3, 7, 4, 8, 5, 10, 9};
const int IP[8] = {2, 6, 3, 1, 4, 8, 5, 7};
const int IP_INV[8] = {4, 1, 3, 5, 7, 2, 8, 6};
const int EP[8] = {4, 1, 2, 3, 2, 3, 4, 1}; // Expansion/Permutation
const int P4[4] = {2, 4, 3, 1};

// S-boxes (4x4)
const int S0[4][4] = {
    {1, 0, 3, 2},
    {3, 2, 1, 0},
    {0, 2, 1, 3},
    {3, 1, 3, 2}};

const int S1[4][4] = {
    {0, 1, 2, 3},
    {2, 0, 1, 3},
    {3, 0, 1, 0},
    {2, 1, 0, 3}};

// ===== Utility Functions =====

// Check if a string contains only '0' and '1'
bool isBinary(const string &s)
{
    for (char c : s)
        if (c != '0' && c != '1')
            return false;
    return true;
}

// Read a binary string of exactly 'bits' characters, with validation
string readBinary(int bits)
{
    string input;
    while (true)
    {
```

```
    cin >> input;
    if (input.length() == bits && isBinary(input))
        return input;
    cout << "Invalid input. Enter exactly " << bits << " bits (0/1): ";
}
}

// Apply a permutation table to an input value.
uint16_t permute(uint16_t input, int in_len, const int perm[], int out_len)
{
    uint16_t output = 0;
    for (int i = 0; i < out_len; i++)
    {
        // Source bit position (0-based from the left)
        int src = perm[i] - 1;
        // Extract that bit (counting from MSB)
        uint16_t bit = (input >> (in_len - 1 - src)) & 1;
        // Place it in the output (MSB first)
        output |= (bit << (out_len - 1 - i));
    }
    return output;
}

// Circular left shift on a 5-bit value
uint8_t leftShift5(uint8_t val, int shifts)
{
    shifts %= 5;
    val &= 0x1F; // keep only lower 5 bits
    return ((val << shifts) | (val >> (5 - shifts))) & 0x1F;
}

// ===== Key Schedule =====
void generateSubkeys(uint16_t key, uint8_t &k1, uint8_t &k2)
{
    // Step 1: P10
    uint16_t permuted = permute(key, 10, P10, 10);

    // Split into two 5-bit halves
    uint8_t left = (permuted >> 5) & 0x1F;
    uint8_t right = permuted & 0x1F;

    // Round 1 (shift 1)
    left = leftShift5(left, 1);
    right = leftShift5(right, 1);
    uint16_t combined = (left << 5) | right;
    k1 = permute(combined, 10, P8, 8); // first subkey

    // Round 2 (shift 2 more → total 3)
    left = leftShift5(left, 2);
    right = leftShift5(right, 2);
    combined = (left << 5) | right;
    k2 = permute(combined, 10, P8, 8); // second subkey
}
```

```
}

// ===== S-box Lookup =====
// Input: 4 bits (packed in low nibble of val)
// Output: 2 bits (result of S-box)
uint8_t sbox(uint8_t val, const int box[4][4])
{
    // Row: bits 1 and 4 (1-based) → bits 4 and 1 (0-based)
    uint8_t row = ((val & 0x8) >> 2) | (val & 0x1);
    // Column: bits 2 and 3
    uint8_t col = (val >> 1) & 0x3;
    return box[row][col] & 0x3;
}

// ===== Feistel Round Function =====
// f(R, SK) : R is 4 bits, SK is 8 bits (subkey)
uint8_t f(uint8_t r, uint8_t sk)
{
    // Expansion/Permutation (4 → 8 bits)
    uint8_t expanded = permute(r, 4, EP, 8);

    // XOR with subkey
    uint8_t xored = expanded ^ sk;

    // Split into two 4-bit halves for the S-boxes
    uint8_t left = (xored >> 4) & 0x0F;
    uint8_t right = xored & 0x0F;

    // Apply S-boxes (each gives 2 bits)
    uint8_t s0_out = sbox(left, S0);
    uint8_t s1_out = sbox(right, S1);

    // Combine (4 bits: s0_out as high 2, s1_out as low 2)
    uint8_t combined = (s0_out << 2) | s1_out;

    // Apply P4 permutation
    return permute(combined, 4, P4, 4);
}

// ===== Encryption =====
uint8_t encrypt(uint8_t plain, uint8_t k1, uint8_t k2)
{
    // Initial Permutation
    uint8_t ip = permute(plain, 8, IP, 8);

    // Split into left and right halves (4 bits each)
    uint8_t left = (ip >> 4) & 0x0F;
    uint8_t right = ip & 0x0F;

    // Round 1
    uint8_t temp = right;
    right = left ^ f(right, k1);
```

```
    left = temp;

    // Round 2
    temp = right;
    right = left ^ f(right, k2);
    left = temp;

    // After the two rounds, swap the halves (as required by Feistel)
    uint8_t combined = (right << 4) | left;

    // Inverse Initial Permutation
    return permute(combined, 8, IP_INV, 8);
}

// ===== Decryption =====
// Same encryption function but with subkeys reversed
uint8_t decrypt(uint8_t cipher, uint8_t k1, uint8_t k2)
{
    return encrypt(cipher, k2, k1);
}

// ===== Main Program =====
int main()
{
    cout << "\n=== SIMPLIFIED DES (SDS) ===\n";

    // Read plaintext (8 bits)
    cout << "Enter 8-bit plaintext: ";
    string plainStr = readBinary(8);

    // Read key (10 bits)
    cout << "Enter 10-bit key: ";
    string keyStr = readBinary(10);

    // Convert binary strings to integers
    uint8_t plain = stoi(plainStr, nullptr, 2);
    uint16_t key = stoi(keyStr, nullptr, 2);

    // Generate subkeys
    uint8_t k1, k2;
    generateSubkeys(key, k1, k2);

    // Encrypt and decrypt
    uint8_t cipher = encrypt(plain, k1, k2);
    uint8_t decrypted = decrypt(cipher, k1, k2);

    // Display results
    cout << "\nSubkey K1: " << bitset<8>(k1);
    cout << "\nSubkey K2: " << bitset<8>(k2);

    cout << "\n\nPlaintext : " << plainStr;
    cout << "\nCiphertext: " << bitset<8>(cipher);
```

```
cout << "\nDecrypted : " << bitset<8>(decrypted);

if (plain == decrypted)
    cout << "\n\nDecryption successful - original plaintext recovered.\n";
else
    cout << "\n\nDecryption failed - something went wrong.\n";

cout << endl;
return 0;
}
```

Code Explanation:

- 1. Permutation Tables:** The code establishes constants like P10, P8, IP, and EP represented as arrays to rearrange bits in accordance with the S-DES standard.
- 2. Bit Manipulation:** The `permute` function serves as a pivotal element in the code, employing bitwise shifts (>> and <<) along with masks (&) to extract bits from designated positions and rearrange them.
- 3. Key Generation:** The `generateSubkeys` function takes a 10-bit key, applies P10, splits the key, performs circular left shifts, and uses P8 to create two 8-bit subkeys (K_1) and (K_2).
- 4. Initial/Inverse Permutation:** Within the `encrypt` function, the process begins with IP to scramble the plaintext and concludes with IP_INV to yield the final ciphertext.
- 5. Feistel Rounds:** The algorithm comprises two rounds. In each round, the right 4 bits are expanded to 8 bits (using EP), XORed with a subkey, and processed through S-boxes.
- 6. S-Box Logic:** The `sbox` function determines the row (by examining the 1st and 4th bits) and the column (using the 2nd and 3rd bits) to retrieve a 2-bit value from either the (S_0) or (S_1) matrices.
- 7. The (f) Function:** This function combines the expansion, S-box substitution, and a P4 permutation to introduce the necessary "confusion" for the encryption process.
- 8. Decryption Symmetry:** The `decrypt` function capitalizes on the Feistel property by calling the `encrypt` function in reverse, applying the subkeys in the order of (K_2) followed by (K_1).
- 9. Input Validation:** The `readBinary` and `isBinary` functions ensure users provide valid 8-bit or 10-bit strings, thus preventing runtime errors during conversion.

2. Output

```
=== SIMPLIFIED DES (SDES) ===  
Enter 8-bit plaintext: 11010111  
Enter 10-bit key: 1010000010  
  
Subkey K1: 10100100  
Subkey K2: 01000011  
  
Plaintext : 11010111  
Ciphertext: 10101000  
Decrypted  : 11010111  
  
Decryption successful - original plaintext recovered.
```

3. Observation

The code illustrates that S-DES, despite its compact bit sizes, successfully emulates the structure of the full DES algorithm. It demonstrates how non-linear substitution (through S-boxes) and linear permutations establish a complex relationship between the plaintext and ciphertext. The symmetry in encryption and decryption is clear through the reversed order of subkey usage.

4. Result

The program effectively processes an 8-bit plaintext alongside a 10-bit key to produce an 8-bit ciphertext. When the decryption process is performed using the same subkeys in reverse order, the output corresponds perfectly with the original plaintext, confirming the accuracy of the Feistel network implementation.

5. Applications

S-DES is mainly used as an educational tool to introduce fundamental concepts of symmetric cryptography without the intricacies associated with 64-bit blocks. It serves as a useful model for grasping "Confusion and Diffusion" and is frequently employed to prototype more extensive Feistel-based algorithms in educational environments.

6. Advantages

- **Simplicity:** The smaller bit lengths facilitate tracing and debugging processes.
- **Resource Efficiency:** It demands minimal memory and computational power, leading to swift execution.
- **Educational Value:** S-DES effectively illustrates the mechanics of subkey generation, S-box substitution, and permutations as used in contemporary block ciphers.

7. Disadvantages

- **Zero Security:** With a key size of only 10 bits, the total number of possible keys is ($2^{10} = 1024$), easily brute-forced by modern computers.

- **Small Block Size:** An 8-bit block size is vulnerable to frequency analysis and pattern recognition.
- **Obsolescence:** S-DES is meant solely for educational purposes and is unsuitable for safeguarding sensitive data.

8. Conclusion

The C++ implementation of S-DES provided here offers a practical model of a symmetric block cipher. While it's regarded as insecure for real-world scenarios due to its limited key space, it effectively demonstrates the inner workings of the DES family. It gives a clear insight into how bit-level permutations and substitutions interact in the realm of data security.

THE END
